

Responsible AI and MLOps Foundations

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية

King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

October 7, 2026

- ▶ Find where bias enters an ML system, audit a model with disaggregated metrics, and explain why the main fairness definitions cannot all hold at once.
- ▶ Explain a model's predictions with permutation importance, SHAP, LIME and Grad-CAM, and know when not to trust the explanation.
- ▶ Document a model with a model card, and place it within SDAIA's AI Ethics Principles and the PDPL.
- ▶ Take a model from experiment to production: reproducible runs, experiment tracking, and a registry with versions and aliases.
- ▶ Deploy and monitor a model: choose a serving pattern, detect drift before the labels arrive, and release behind shadow and canary tests.
- ▶ Make a model smaller and faster with quantisation, pruning and distillation, and gate the compressed version like any other release.

Today: Two Lectures, Two Labs

Block	Content
Lecture 1	Part 1: Bias and Fairness Part 2: Interpretability Part 3: Documentation and Governance
Lecture 2	Part 4: From Experiment to Production Part 5: Lifecycle and Versioning Part 6: Deployment and Monitoring Part 7: Compression and Efficient Inference
Lab 1	Quantisation: by hand, with ONNX Runtime, then on a YOLO11 detector, live on a webcam
Lab 2	Pruning and distillation, then ship one through a release gate and an MLflow registry

Optional labs, on the course site under Day 5

A Responsible AI audit (fairness metrics, SHAP and LIME, a model card), and an end-to-end MLOps loop (track, tune, deploy, monitor, roll back).

1. Part 1: Bias and Fairness
2. Part 2: Interpretability
3. Part 3: Documentation and Governance
4. Part 4: From Experiment to Production
5. Part 5: Lifecycle and Versioning
6. Part 6: Deployment and Monitoring
7. Part 7: Compression and Efficient Inference
8. Summary
9. References

The system. COMPAS is a commercial risk-assessment tool (Northpointe, now Equivant) that scores a criminal defendant's likelihood of re-offending on a 1–10 scale. US courts have used such scores to inform bail, sentencing, and parole decisions.

The analysis. ProPublica journalists obtained scores for more than 7,000 people arrested in Broward County, Florida in 2013–2014 and checked, two years later, who actually re-offended.

	White	Black
Labelled higher risk, but did <i>not</i> re-offend	23.5%	44.9%
Labelled lower risk, yet <i>did</i> re-offend	47.7%	28.0%

- ▶ Overall, 61% of those scored likely to re-offend were arrested for some crime within two years.
- ▶ Of those predicted to commit a *violent* crime, 20% did.

J. Angwin, J. Larson, S. Mattu and L. Kirchner, "Machine Bias," *ProPublica*, 23 May 2016, [propublica.org](https://www.propublica.org/article/machine-bias-risk-assessments-in-criminal-sentencing).

Case 1: The Disagreement That Followed

Northpointe replied that its score was **equally accurate for both groups**: among defendants given the same score, roughly the same fraction re-offended regardless of race.

Both sides were reporting true numbers

ProPublica measured **error rates** conditioned on the true outcome (who got wrongly flagged). Northpointe measured **calibration** conditioned on the score (what a score means). These are different quantities.

- ▶ Chouldechova (2017) and Kleinberg, Mullainathan and Raghavan (2016) showed independently that when the two groups have **different base rates** of the outcome, a score cannot satisfy both criteria at once, except in degenerate cases.
- ▶ So this was not a bug that better engineering would fix. It was a **choice between incompatible definitions of fair** — and nobody had made it explicitly.

We return to this result in Section 4.

Case 2: A Resume-Screening Tool (Reuters reporting, 2018)

Note on evidence

This case comes from **news reporting based on anonymous sources**, not from a published study. Treat the mechanism as illustrative and the details as journalistic.

According to Reuters, a team at Amazon built experimental tools from 2014 that scored job applicants one to five stars. By 2015 the team found the system was not rating candidates for software-developer roles in a gender-neutral way.

- ▶ The models were trained on resumes submitted to the company over roughly a decade — a pool that was predominantly male in technical roles.
- ▶ The reporting states the system penalised resumes containing the word “women’s” and downgraded graduates of two all-women’s colleges.

- ▶ Amazon said the tool was never used by recruiters to evaluate candidates, and the project was abandoned.

The mechanism, stated plainly

The label was not “a good engineer.” It was “a person this company hired before.” Removing the gender field does not help, because the model finds **proxies**. The learning worked; the **target definition** was the defect.

J. Dustin, “Amazon scraps secret AI recruiting tool that showed bias against women,” *Reuters*, 10 October 2018.

Case 3: Gender Shades — Disaggregated Error Rates (2018)

Buolamwini and Gebru audited three **commercial** gender-classification APIs on the *Pilot Parliaments Benchmark* (PPB): 1,270 parliamentarians from three African and three European countries, labelled by Fitzpatrick skin type and by binary gender.

Classifier	All	Darker female	Darker male	Lighter female	Lighter male
Microsoft	6.3%	20.8%	6.0%	1.7%	0.0%
Face++	10.0%	34.5%	0.7%	9.8%	0.8%
IBM	12.1%	34.7%	12.0%	7.1%	0.3%

Error rate = 1 – true positive rate, evaluated April–May 2017.

- ▶ The aggregate column looks acceptable. The intersectional columns do not.
- ▶ Marginal slices hide it: an 8.1%–20.6% gap by gender alone, 11.8%–19.2% by skin type alone.
- ▶ Maximum reported error-rate gap between the best- and worst-classified subgroups: **34.4%**.

Two lessons. (1) Slice by every group *and by combinations*, and report the worst slice with its sample size. (2) Quote the study for what it measured — binary gender classification, one benchmark, three APIs, one point in time — not as “face recognition does not work.”

J. Buolamwini and T. Gebru, “Gender Shades: Intersectional Accuracy Disparities in Commercial Gender Classification,” *PMLR* 81:77–91, FAT* 2018, Table 4 and Section 4.1.

First: “Bias” Means Three Different Things

Statistical bias

$$\mathbb{E}[\hat{\theta}] - \theta \neq 0.$$

The estimator is systematically off. Neutral term; half of the bias–variance trade-off.

Inductive bias

The assumptions a model class makes.

Convolutions assume locality; linear models assume additivity. **Necessary** — without it there is no generalisation.

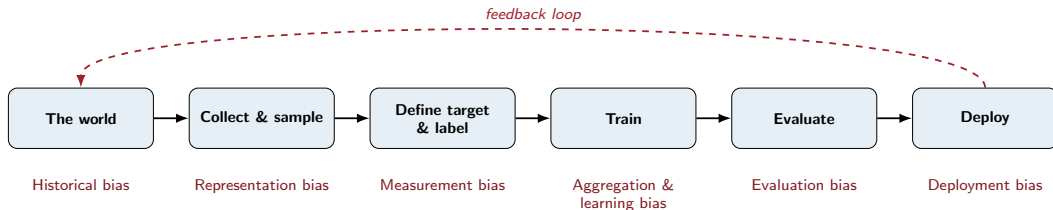
Societal / harmful bias

Systematically different quality of service or outcome across groups of people.

This lecture’s subject.

The three are unrelated. A model can be statistically unbiased and still harmful; a strong inductive bias can either mask or expose a societal one.

Bias Enters at Every Stage, Not Just “The Data”



Taxonomy after H. Suresh and J. V. Gutttag, “A Framework for Understanding Sources of Harm throughout the Machine Learning Life Cycle,” EAAMO 2021, [arXiv:1901.10002v5](#).

Historical bias

Present even with perfect sampling and perfect measurement, because the world the data records is itself unequal.

- ▶ An image search for “CEO” that reflects who currently holds the job.
- ▶ A loan model trained on approvals from an era of discriminatory lending.

More data does not fix this. It is a question about the objective, not the sample.

Representation bias

The development sample under-represents part of the use population, so the model generalises poorly there.

- ▶ A dermatology model trained mostly on light skin.
- ▶ A speech model trained on one accent.
- ▶ Gender Shades: existing face benchmarks were 79.6% and 86.2% lighter-skinned subjects.

More *targeted* data does help — but you must first know the composition of your data.

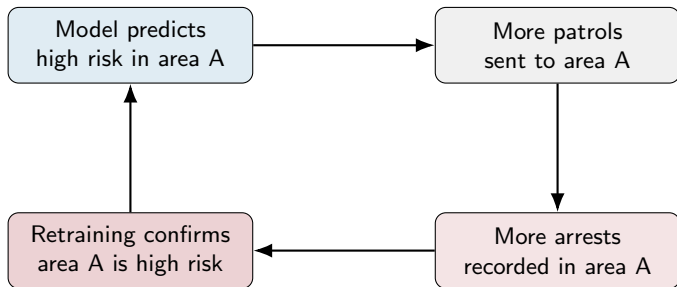
You almost never observe the thing you care about. You observe something correlated with it.

What you want	What you record	What the gap encodes
Health need	Health-care spending	Unequal access to care
Committing a crime	Being arrested	Unequal policing intensity
Job performance	Was hired / was promoted	Past hiring decisions
Creditworthiness	Repaid a loan they were <i>granted</i>	Who was offered credit
Content quality	Clicks and watch time	Engagement, not quality

The one question to ask on every project

“My label is X. What exactly had to happen in the world for X to be recorded, and does that differ by group?”

Feedback Loops: The Model Changes Its Own Training Data

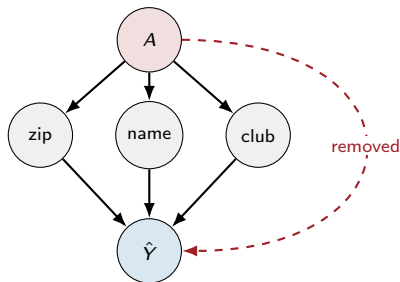


- ▶ The loop is self-confirming: the prediction generates the evidence for itself.
- ▶ Same shape in recommenders (you only observe clicks on what you showed), in credit (you only observe repayment for approved applicants), and in hiring.
- ▶ **Detection:** hold out a slice from the policy, or log what the model *suppressed*, not just what it surfaced.

“Just Drop the Sensitive Attribute” Does Not Work

Fairness through unawareness: remove race, gender, age from the feature set and declare the model neutral.

- ▶ Real features are **correlated** with the removed attribute: postcode, first name, school, device type, browsing history, purchase categories.
- ▶ A capable model *reconstructs* the attribute from the proxies. Removing the column removes your ability to **measure** the disparity, not the disparity itself.
- ▶ The Amazon reporting is exactly this: no gender field, gendered signal.
- ▶ Corollary: you often need the sensitive attribute — held securely and used only for auditing — in order to check for harm.



Cutting the direct edge leaves every indirect path intact.

One confusion matrix per group. Every definition below is a constraint tying two cells of these tables together across groups.

		$\hat{Y} = 1$	$\hat{Y} = 0$
Group A	$Y = 1$	TP	FN
	$Y = 0$	FP	TN

- ▶ **Selection rate** $= \Pr(\hat{Y} = 1)$
- ▶ **TPR / recall** $= \Pr(\hat{Y} = 1 \mid Y = 1)$
- ▶ **FPR** $= \Pr(\hat{Y} = 1 \mid Y = 0)$
- ▶ **FNR** $= 1 - \text{TPR}$
- ▶ **PPV / precision** $= \Pr(Y = 1 \mid \hat{Y} = 1)$
- ▶ **Prevalence** $p = \Pr(Y = 1)$

A fairness *audit* is: build this table per group, then compare a chosen cell across groups. The hard part is choosing which cell.

Definition 1: Demographic Parity (Independence)

Demographic parity / statistical parity

$$\Pr(\hat{Y} = 1 \mid A = A) = \Pr(\hat{Y} = 1 \mid A = B)$$

The model selects the same *proportion* of each group.

When it is the right question

- ▶ Allocating an opportunity where you do not trust the label (advertising, outreach, shortlisting).
- ▶ You believe the historical outcome Y is itself contaminated.
- ▶ A widely used convention: a selection-rate ratio below 0.8 between groups is treated as a gap worth investigating. Most audit tooling flags it by default.

Where it goes wrong

- ▶ It ignores Y entirely. A model can satisfy it by selecting qualified people from one group and arbitrary people from the other.
- ▶ If the true base rates genuinely differ, enforcing it forces errors.
- ▶ Measured as a **ratio** (disparate impact) or a **difference** — report which.

Definition 2: Equalised Odds (Separation)

Equalised odds — Hardt, Price and Srebro (2016)

$$\Pr(\hat{Y} = 1 \mid Y = y, A = A) = \Pr(\hat{Y} = 1 \mid Y = y, A = B) \quad \text{for } y \in \{0, 1\}$$

Equal **TPR** and equal **FPR** across groups.

Equal opportunity — the one-sided version

Only $\Pr(\hat{Y} = 1 \mid Y = 1)$ must match: among people who *would* have succeeded, each group has the same chance of being selected.

- ▶ This is the ProPublica criterion: their complaint was unequal FPR and FNR.
- ▶ Use it when the burden of a **mistake** is what matters — a wrongly denied loan, a wrongly flagged defendant, a missed diagnosis.
- ▶ It conditions on Y , so it inherits whatever bias is in the label. If Y is “was arrested,” equalised odds equalises errors against a biased ground truth.

Definition 2: Equalised Odds (Separation) (cont.)

M. Hardt, E. Price and N. Srebro, "Equality of Opportunity in Supervised Learning," NeurIPS 2016,
[arXiv:1610.02413v1](#).

Definition 3: Calibration and Predictive Parity (Sufficiency)

Calibration within groups

$$\Pr(Y = 1 \mid R = r, A = A) = \Pr(Y = 1 \mid R = r, A = B) = r$$

A score of 0.7 means a 70% chance for everyone, whatever their group. **Predictive parity** is the thresholded version: equal PPV across groups.

- ▶ This is the criterion the COMPAS vendor pointed to, and it is a genuinely important one: without it, a human reading the score is being misled about what it means.
- ▶ Well-trained probabilistic models are often *approximately* calibrated within groups by default — which is precisely why the conflict below is unavoidable rather than exotic.
- ▶ Check it with a **reliability diagram per group**, not with a single number.

Chouldechova (2017), Equation (2.6)

$$\text{FPR} = \frac{p}{1-p} \cdot \frac{1 - \text{PPV}}{\text{PPV}} \cdot (1 - \text{FNR})$$

with p the prevalence $\Pr(Y = 1)$ in that group.

Fix PPV to be equal across two groups. If $p_A \neq p_B$, the right-hand side differs, so FPR and FNR **cannot both** be equal. Predictive parity and error-rate balance are incompatible whenever base rates differ.

- ▶ Kleinberg, Mullainathan and Raghavan (2016) prove the same tension for calibrated *scores*: except in degenerate cases (equal base rates, or a perfect predictor), calibration and balance for both classes cannot hold together.
- ▶ Nothing about this depends on the model, the features, or the amount of data. It is arithmetic.

You Cannot Have All Three (cont.)

A. Chouldechova, "Fair prediction with disparate impact," *Big Data* 5(2), 2017, [arXiv:1703.00056v1](#). J. Kleinberg, S. Mullainathan and M. Raghavan, "Inherent Trade-Offs in the Fair Determination of Risk Scores," ITCS 2017, [arXiv:1609.05807v2](#).

Two groups, 100 people each. Prevalence $\Pr(Y = 1)$ in the population: $p_A = 0.5$, $p_B = 0.3$. A model that is **equally accurate within each group** (TPR = 0.8, TNR = 0.8 for both):

	$Y = 1$	$Y = 0$	TP	FP	PPV	FPR
Group A ($p = 0.5$)	50	50	40	10	$40/50 = \mathbf{0.80}$	$10/50 = \mathbf{0.20}$
Group B ($p = 0.3$)	30	70	24	14	$24/38 = \mathbf{0.63}$	$14/70 = \mathbf{0.20}$

- ▶ Error rates are balanced (equal TPR and FPR) — but a positive prediction means something different in each group (0.80 vs 0.63).
- ▶ Push PPV to match and the error rates separate. There is no threshold that fixes both.
- ▶ **Also note:** the selection rate is 50/100 vs 38/100, so demographic parity fails as well.

So Which Definition Do You Use?

This is a decision about **harms**, not a hyperparameter search. Four questions, in order:

1. **Who is affected, and what is the harm?** Being wrongly *denied* something (allocative) or being wrongly *flagged* (punitive) point to different error rates.
2. **Do you trust the label Y ?** If the label encodes past discrimination, definitions that condition on Y inherit it — lean towards independence.
3. **Is the score read by a human?** If yes, calibration within groups is close to mandatory, or you are misinforming the decision-maker.
4. **Can you measure it?** A criterion you cannot estimate — no group labels, or slices too small — is a wish, not a criterion (see the next section).

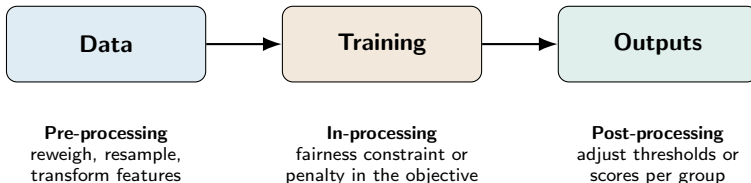
Write the chosen criterion, and the criteria you consciously gave up, into the model card. An unstated choice is still a choice.

One number for the test set, then the same number for every slice you can define.

Slice	n	Selection rate	TPR	FPR	PPV
Overall	10,000	0.24	0.78	0.11	0.71
Group A	8,200	0.26	0.81	0.10	0.73
Group B	1,800	0.14	0.62	0.13	0.58
Group B, age < 30	420	0.11	0.49	0.14	0.51
Group B, age $\geq$ 60	180	0.09	0.44	0.12	0.47

Illustrative numbers, constructed for this slide.

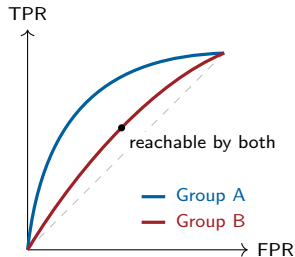
- ▶ The overall row is fine. Group B is not, and the intersectional rows are worse — the pattern from Gender Shades, reproduced in miniature.
- ▶ Always print n beside the metric. Always print the **worst slice** in the summary, not just the mean.
- ▶ In `scikit-learn`, this is a `groupby` plus `classification_report`; in `fairlearn` it is `MetricFrame`.



- ▶ **Access decides the option.** Third-party model behind an API: post-processing only. Your own training loop: all three.
- ▶ **None of them repair a bad label or a missing population.** Those are data problems, and mitigation applied on top will only redistribute the error.

Hardt et al. (2016) show that if you keep the score R and choose a **separate threshold per group**, you can achieve equalised odds — and they give the optimal choice.

- ▶ Each group's ROC curve gives the achievable (FPR, TPR) pairs.
- ▶ Equalised odds requires landing on the **same point** for both groups, so the feasible set is the intersection.
- ▶ The best you can do is the intersection point that maximises utility — which is generally worse than either group's own optimum.



Schematic drawn for this slide, after the construction in Hardt et al. (2016).

Before you reach for this

Per-group thresholds are the most **visible** intervention in this section: two people with the same score get different outcomes, and you will have to say so out loud. In many settings that alone rules it out. Treat it as a decision to be agreed and recorded, not as a parameter to be tuned — the statistically optimal fix is not always the acceptable one.

Expect to give something up

- ▶ Constraining a metric usually costs some accuracy. Report the curve (disparity vs accuracy), not one point — it turns an argument into a decision.
- ▶ Sometimes the cost is near zero, which is worth discovering.
- ▶ “Levelling down” — degrading the strong group to close the gap — satisfies the metric and helps nobody. Say explicitly whether it is acceptable.

After deployment

- ▶ Re-run the disaggregated report on production traffic on a schedule.
- ▶ Alert on disparity, not only on accuracy drift.
- ▶ Keep a channel for people to contest a decision, and log contested cases — they are your best source of failure modes.

Libraries

- ▶ fairlearn (metrics, reductions, threshold optimiser)
- ▶ AIF360 (a wide catalogue of algorithms)
- ▶ aequitas (audit reports)

Optional lab: disaggregate on UCI Adult, show the parity/odds tension, apply threshold post-processing, and measure what it cost.

- ▶ **Debugging.** By far the most common real use. Explanations expose leakage, shortcut features, and broken preprocessing that accuracy alone hides.
- ▶ **Trust and adoption.** A clinician or underwriter who cannot see the reasoning will not use the tool — or will over-trust it.
- ▶ **Recourse.** Telling a rejected applicant *what to change* is a different requirement from telling an engineer what the model used.
- ▶ **Accountability.** When a decision is challenged, somebody has to be able to say why the model produced it. “The network decided” is not an answer anyone accepts twice.
- ▶ **Science.** In a scientific application the model *is* the hypothesis, and what it uses is the result you wanted.

Each purpose demands a different artefact. Decide the purpose **before** choosing the method.

Axis		
Intrinsic vs post-hoc	The model is readable as it stands: linear/logistic regression, a shallow tree, a rule list, a GAM.	An external procedure explains a fitted black box: permutation importance, SHAP, LIME, Grad-CAM.
Global vs local	What does the model do <i>in general</i> ? Feature importances, partial dependence.	Why <i>this</i> prediction for <i>this</i> row? SHAP values, LIME, counterfactuals.
Model-specific vs model-agnostic	Uses the internals: tree paths, gradients, attention, TreeSHAP.	Only needs <code>predict()</code> : permutation importance, KernelSHAP, LIME.

Model-agnostic buys generality at the price of speed and, often, fidelity. Model-specific is faster and usually more faithful, but locks you to a model family.

Rudin (2019)

For high-stakes decisions, prefer a model that is interpretable by construction over a black box plus a post-hoc explanation — because the explanation is an **approximation** of the model, and an approximation can be wrong in exactly the cases you care about.

- ▶ On many tabular problems, a well-regularised logistic regression, a depth-4 tree, or a generalised additive model is within a point or two of a gradient-boosted ensemble.
Measure the gap before assuming you need the black box.
- ▶ If the gap is real and worth it, take the black box — but then the explanation needs its own validation, exactly as the model does.
- ▶ Interpretability is not a property of the model alone. A linear model with 4,000 engineered features is not interpretable; a depth-3 tree over four meaningful features is.

C. Rudin, “Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead,” *Nature Machine Intelligence* 1:206–215, 2019, [arXiv:1811.10154v3](https://arxiv.org/abs/1811.10154v3).

Recipe

1. Measure the model's score on the validation set.
2. Shuffle one feature column, breaking its relationship with the target.
3. Re-score. The **drop** is that feature's importance.
4. Repeat several times and average; then restore the column and move on.

Why start here

- ▶ Model-agnostic, ten lines of code, needs only `predict()`.
- ▶ Measures importance **for the metric you care about**.
- ▶ Computed on *held-out* data, so it reflects generalisation, not fit.

Where it misleads

- ▶ **Correlated features split the credit**, and each looks unimportant because the other covers for it.
- ▶ Shuffling creates impossible rows (age 8, income \$200k), so you evaluate the model off its data manifold.
- ▶ It is global only — it says nothing about a single prediction.

A Trap: Built-in Tree “Feature Importance”

`RandomForest.feature_importances_` is **mean decrease in impurity** (MDI), accumulated on the *training* data.

- ▶ It is biased towards **high-cardinality** features: a continuous feature or a random ID column offers many split points, so it looks informative even when it is pure noise.
- ▶ It is computed on training data, so it rewards features the model used to overfit.
- ▶ Classic demonstration: add a column of random integers to your dataset and it can outrank real predictors in MDI — while permutation importance on held-out data correctly gives it roughly zero.

Rule: never report `feature_importances_` as “the model’s feature importance” without saying which definition you used. Prefer permutation importance or SHAP on held-out data.

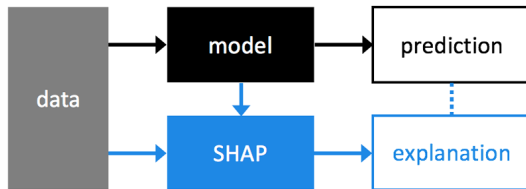
The analogy. A team of players cooperates and earns a payout. How much of it does each player deserve? Here the “players” are the features of one row, and the “payout” is $f(x) - \mathbb{E}[f(X)]$: how far this prediction sits from the average prediction.

Shapley value (Shapley, 1953)

Consider adding feature i to a coalition S of features already present, and record the change in the prediction. The Shapley value ϕ_i is the **average of that change over all possible orderings** in which features could be added:

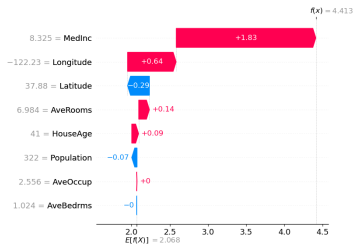
$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|! (|N| - |S| - 1)!}{|N|!} \left[v(S \cup \{i\}) - v(S) \right]$$

- ▶ The averaging over orderings is what makes it symmetric: no feature is privileged by being “first”.
- ▶ There are $2^{|N|}$ subsets, so the exact computation is exponential. Everything practical in SHAP is an efficient approximation of this quantity.



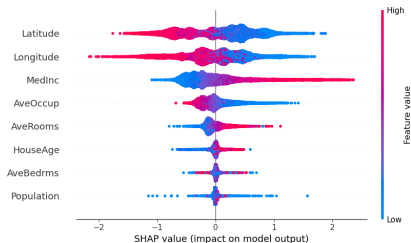
Source: shap library documentation artwork, github.com/shap/shap (docs/artwork/shap_diagram.png).

- ▶ SHAP does not change the model. It takes the **fitted model** plus a **background dataset** and produces one attribution per feature per row.
- ▶ The background dataset defines $\mathbb{E}[f(X)]$ — the reference the explanation is measured against. **Change the background and every number changes**, so record which rows you used.
- ▶ A useful phrasing for stakeholders: *“compared with a typical applicant, this applicant’s income pushed the score up by 0.31.”*



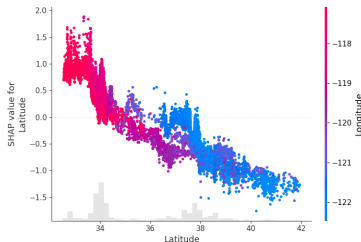
Source: shap library documentation artwork, [github.com/shap/shap](https://github.com/shap/shap/blob/master/docs/artwork/california_waterfall.png) (docs/artwork/california_waterfall.png); model trained on the California Housing dataset, target in units of \$100k.

- ▶ Read bottom to top: start at the base value $\mathbb{E}[f(X)] = 2.068$, add each feature's contribution, arrive at this row's prediction $f(x) = 4.413$.
- ▶ The grey number on the left is the feature's actual value for this row.
- ▶ This is the artefact to show a domain expert. One row, one story, and it adds up.



Source: shap library documentation artwork, github.com/shap/shap (docs/artwork/california_beeswarm.png).

- ▶ One dot per row per feature; horizontal position is the SHAP value, colour is the **feature value** (red high, blue low). Features are ordered by mean $|\phi|$.
- ▶ Read the **direction**: for MedInc, red dots sit on the right, so higher median income pushes the predicted price up — monotone and sensible.
- ▶ Read the **spread**: a feature with a wide swarm matters a lot for some rows and not at all for others. A bar chart of mean $|\phi|$ would have hidden that.



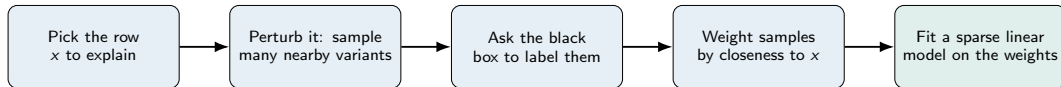
Source: shap library documentation artwork, github.com/shap/shap (docs/artwork/california_scatter.png).

- ▶ x-axis: the feature's value. y-axis: its SHAP value. This is the model's learned response to that feature, without assuming it is linear or monotone.
- ▶ Colouring by a second feature exposes **interactions**: here the vertical spread at a given latitude is explained by longitude — the model has learned geography, not a one-dimensional rule.
- ▶ Sharp steps are informative: they are the split points of the underlying trees.

Which SHAP Estimator?

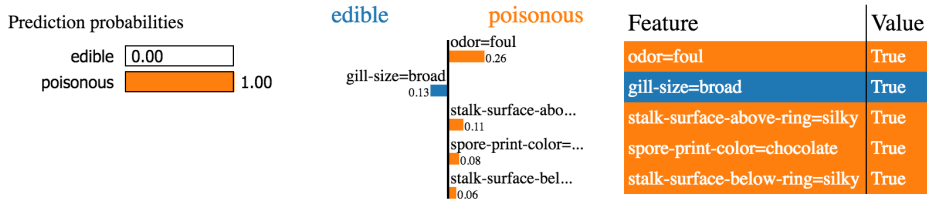
Estimator	Applies to	Notes
TreeSHAP	Trees, random forests, gradient boosting (XGBoost, LightGBM, CatBoost)	Exact, in polynomial time, by exploiting the tree structure. Fast enough for a whole dataset. Use this whenever you can.
KernelSHAP	Anything with <code>predict()</code>	Weighted local linear regression over sampled coalitions. Model-agnostic but slow ; sample a few hundred background rows and a subset of instances.
DeepSHAP / Gradient	Neural networks	Uses backpropagation and a DeepLIFT-style approximation; much cheaper than KernelSHAP.
Linear	Linear models	Closed form: $\phi_i = w_i(x_i - \mathbb{E}[X_i])$ under feature independence.

Polynomial-time exact tree attribution: S. M. Lundberg et al., “From local explanations to global understanding with explainable AI for trees,” *Nature Machine Intelligence* 2:56–67, 2020.



- ▶ The coefficients of that little linear model *are* the explanation.
- ▶ “Perturb” is domain-specific: for text, delete words; for images, switch superpixels on and off; for tabular data, sample around the row.
- ▶ The surrogate is deliberately **sparse** (a handful of features) so a human can read it. Complexity is traded for legibility, on purpose.

M. T. Ribeiro, S. Singh and C. Guestrin, “‘Why Should I Trust You?’: Explaining the Predictions of Any Classifier,” KDD 2016, [arXiv:1602.04938v3](https://arxiv.org/abs/1602.04938v3).



Source: lime library documentation, github.com/marcotcr/lime (doc/images/tabular.png); mushroom edibility classifier.

- ▶ Left: the predicted class probabilities. Middle: signed contributions of the top features. Right: the actual feature values for this row.
- ▶ The explanation is in terms of **discretised bins** (odor=foul), which is why it reads well to a non-specialist — and why the numbers depend on how you binned.
- ▶ Note the contrast with SHAP: LIME's weights are coefficients of a local surrogate. They do **not** sum to the prediction.

Prediction probabilities



atheism

christian

Posting
0.15
Host
0.14
NNTP
0.11
edu
0.04
have
0.01
There
0.01

Text with highlighted words

From: johnchad@triton.unm.edu (jchadwic)
Subject: Another request for Darwin Fish
Organization: University of New Mexico, Albuquerque
Lines: 11
NNTP-Posting-Host: triton.unm.edu

Hello Gang,

There have been some notes recently asking where to obtain the DARWIN fish.

This is the same question I **have** and I **have** not seen an answer on the net. If anyone has a contact please post on the net or email me.

Source: lime library documentation, github.com/marcotcr/lime (doc/images/twoclass.png); 20 Newsgroups
atheism-vs-christianity classifier.

- ▶ The classifier reports 0.58 for “atheism.” The explanation shows the evidence: Posting, Host, NNTP, edu — **email header artefacts**, not content.
- ▶ This is the single most valuable use of local explanations: a high-accuracy model that is right for a reason that will not survive contact with new data.
- ▶ You would never have found it from the accuracy score.

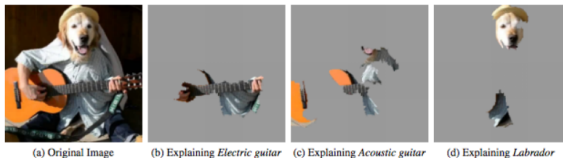


Figure 4: Explaining an image classification prediction made by Google's Inception network, highlighting positive pixels. The top 3 classes predicted are "Electric Guitar" ($p = 0.32$), "Acoustic guitar" ($p = 0.24$) and "Labrador" ($p = 0.21$)

Source: Figure 4 of M. T. Ribeiro, S. Singh and C. Guestrin, "Why Should I Trust You?: Explaining the Predictions of Any Classifier," KDD 2016 ([arXiv:1602.04938v3](https://arxiv.org/abs/1602.04938v3)), as reproduced in the [lime](#) repository.

- ▶ The image is segmented into superpixels; perturbation switches groups of them off.
- ▶ Each panel shows the superpixels supporting a *different* predicted class — the explanation is always **relative to a target class**, never absolute.
- ▶ Useful check: if the supporting region is the background rather than the object, you have found a dataset artefact.

- ▶ **Correlated features.** Both methods perturb features independently, producing rows that could not exist. Attribution then gets spread arbitrarily among correlated features, and can be assigned to a feature the model barely uses. Group correlated features and attribute to the group.
- ▶ **The background / neighbourhood is a choice.** SHAP values are relative to the background dataset; LIME weights depend on the kernel width and the perturbation scheme. Report what you used — otherwise the numbers are not reproducible.
- ▶ **Instability.** Re-running LIME with a different seed can reorder the top features. Always run it several times and check the explanation is stable before you show it to anyone.
- ▶ **Explanations can be attacked.** Slack et al. (2020) built classifiers that behave in a biased way on real data but detect the synthetic perturbation distributions used by LIME and SHAP, and answer innocuously there — so the explanation hides the bias.

Optional lab: run SHAP and LIME on the same tabular model, and watch LIME's top features change from one run to the next.

D. Slack, S. Hilgard, E. Jia, S. Singh and H. Lakkaraju, "Fooling LIME and SHAP: Adversarial Attacks on Post hoc Explanation Methods," AIES 2020, [arXiv:1911.02508v2](https://arxiv.org/abs/1911.02508v2).

What SHAP answers

“How does *this fitted function* distribute its output across the inputs I gave it?”

A statement about the model. Correct as far as it goes.

What people hear

“Which factors *cause* this outcome, and what should this person change?”

A causal statement. Not supported by the attribution.

- ▶ Kumar et al. (2020) argue that the mathematical appeal of Shapley values does not transfer to the human goals people bring to explanations, and that closing the gap requires causal assumptions the method does not make.
- ▶ If the user needs **recourse** (“what do I change?”), the right tool is a **counterfactual explanation** — the nearest feasible input that flips the decision — not a feature attribution.
- ▶ **Not unique, and not a defence.** Different methods, backgrounds and seeds give different attributions for the same prediction; a reviewer finding an explanation satisfying is not evidence the model is correct.
- ▶ **Rule of thumb:** use explanations to **generate hypotheses**, then test each one by intervention — an ablation, a re-fit without the feature, a fresh slice.

I. E. Kumar, S. Venkatasubramanian, C. Scheidegger and S. Friedler, “Problems with Shapley-value-based explanations as feature importance measures,” ICML 2020, [arXiv:2002.11097v2](https://arxiv.org/abs/2002.11097v2).

- ▶ There is no meaningful “feature” called pixel (117, 204). Attribution must be summarised over **regions**, or it is unreadable.
- ▶ There are hundreds of thousands of inputs, so coalition-based methods are far too expensive without a gradient shortcut.
- ▶ The inputs are strongly correlated by construction — neighbouring pixels, adjacent tokens — which is exactly the regime where independent perturbation misleads.

Vanilla gradient saliency (Simonyan, Vedaldi & Zisserman, 2014)

Take the score $s_c(x)$ for class c , backpropagate to the input, and display $|\partial s_c / \partial x|$ as an image.

- ▶ Interpretation: the pixels whose small changes would move the class score most. In practice raw gradients are **visually noisy** — speckle rather than an object.
- ▶ Common repairs: **SmoothGrad** (average the gradient over noisy copies of the input) and **Integrated Gradients** (accumulate gradients along a path from a baseline to the input, restoring an additivity property analogous to SHAP's).

- ▶ Each needs a **baseline** or a noise level. As with SHAP's background, the choice changes the picture.

K. Simonyan, A. Vedaldi and A. Zisserman, "Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps," ICLR 2014 Workshop, [arXiv:1312.6034v2](#). M. Sundararajan, A. Taly and Q. Yan, "Axiomatic Attribution for Deep Networks," ICML 2017, [arXiv:1703.01365v2](#).

The idea, in three steps

1. Take the feature maps of the **last convolutional layer** — they still carry spatial position, but encode semantics.
 2. Weight each map by the average gradient of the target class score with respect to that map (“how much does this channel matter for this class”).
 3. Sum the weighted maps, apply ReLU to keep positive evidence, and upsample to the image size.
- ▶ Coarse but **class-discriminative**: change the target class and the heatmap moves.
 - ▶ Works on any CNN without retraining; ViT variants exist.
 - ▶ **Practitioner’s use**: check that the heatmap sits on the object, and not on a watermark, a ruler, a hospital tag, or the background.



Same image, same network, two target classes: *dog* (left), *cat* (right).

Source: example outputs from the [pytorch-grad-cam](#) library (ResNet-50). Method: R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh and D. Batra, “Grad-CAM,” ICCV 2017, [arXiv:1610.02391v3](#).

Adebayo et al. (2018)

Two tests any saliency method should pass:

- ▶ **Model randomisation:** progressively randomise the trained weights. The explanation should change.
- ▶ **Data randomisation:** retrain on randomly permuted labels. The explanation should change.

They find that **some existing saliency methods are independent both of the model and of the data generating process** — producing similar-looking maps regardless.

- ▶ Such a method is closer to an **edge detector** than to an explanation. It looks convincing because the object outline is where the edges are.
- ▶ Practical consequence: run the randomisation test yourself on the method you plan to ship. It costs one retraining run.

J. Adebayo, J. Gilmer, M. Muelly, I. Goodfellow, M. Hardt and B. Kim, "Sanity Checks for Saliency Maps," NeurIPS 2018, [arXiv:1810.03292v3](https://arxiv.org/abs/1810.03292v3).

A model that ships without documentation forces every downstream user to rediscover its limits — usually in production.

Model card

Travels **with the trained model**. Answers: what is it for, who is it for, where does it fail, and how well does it work *per group*.

Mitchell et al., FAT* 2019.

Datasheet

Travels **with the dataset**. Answers: why was it collected, from whom, with what consent, how was it cleaned, and what should it not be used for.

Gebru et al., CACM 2021.

Both are borrowed ideas

A model card is a **datasheet for a component**, as in electronics; a dataset datasheet is the same idea one step upstream. Neither is a research artefact — they are engineering hygiene, and increasingly they are what a customer or a partner asks to see before they will build on your model.

Section	Content
Model details	Owner, date, version, model type, training algorithm, licence, citation, contact.
Intended use	Primary intended uses and users; out-of-scope uses stated explicitly.
Factors	Groups, instrumentation, and environments the performance is expected to vary over.
Metrics	Which measures, which decision threshold, and how uncertainty was estimated.
Evaluation data	Which datasets, why they were chosen, and how they were preprocessed.
Training data	Distribution over the same factors, as far as it can be disclosed.
Quantitative analyses	Results unitary (per factor) and intersectional (per combination).
Ethical considerations	Sensitive data, risks to people, mitigations applied, known problematic uses.
Caveats & recommendations	Untested groups, open concerns, what testing should happen next.

M. Mitchell, S. Wu, A. Zaldivar, P. Barnes, L. Vasserman, B. Hutchinson, E. Spitzer, I. D. Raji and T. Gebru, “Model Cards for Model Reporting,” FAT* 2019, Section 4, [arXiv:1810.03993v2](https://arxiv.org/abs/1810.03993v2).

Model card — loan-default-v3

Model details. Gradient-boosted trees (LightGBM, 400 trees, depth 6). Owner: Credit Risk team. Version 3.1, trained 2026-05-14 on applications from 2021–2025. Contact: risk-ml@example.org.

Intended use. Produces a default-probability score used to *prioritise manual review* of personal-loan applications between \$1k and \$25k in Market X. Primary users: underwriting analysts.

Out of scope. Not for automatic decline without human review. Not for business loans, mortgages, or other markets. Not for pricing. Not valid for applicants under 21 (fewer than 300 in training).

Factors. Reported over: age band, gender, region, income decile, thin-file vs thick-file credit history, and the region $\times$ thin-file intersection.

Metrics. AUC and recall at the operating threshold 0.31, chosen to keep the manual-review queue at 20% of volume. 95% intervals from 1,000 bootstrap resamples.

Fairness criterion adopted. Equal opportunity on the review decision (equal TPR across region and thin-file status). Demographic parity was **not** targeted, because base default rates differ materially by region; this trade-off was signed off by the Model Risk Committee on 2026-05-02.

Model card — loan-default-v3 (continued)

Quantitative analyses.

Slice	<i>n</i>	AUC	TPR @ 0.31	FPR @ 0.31
Overall	42,300	0.81	0.74	0.18
Region North	31,500	0.82	0.76	0.17
Region South	10,800	0.77	0.66	0.21
Thin file	6,100	0.71	0.58	0.24
South × thin file	1,450	0.68	0.52	0.27

Ethical considerations. Region correlates with an attribute protected in Market X. Region is not an input, but postcode-derived features are; a proxy audit is run quarterly. A wrongly high score delays access to credit, so recall on the worst slice is the primary fairness metric.

Caveats and recommendations. The worst slice is 3.4% of volume and is **materially worse** than overall — route it to a specialist queue rather than the standard one. Not validated after a policy change to income verification (2026-03). Re-audit on 2026-11-14 or after any change to the acquisition channel.

Seven groups of questions, answered by whoever created the dataset:

1. **Motivation.** Why was it created, by whom, funded by whom?
2. **Composition.** What is an instance? How many? What is missing? Does it contain data about people, or confidential/sensitive content?
3. **Collection process.** How was it acquired, by whom, over what period? Were the people informed, and did they consent?
4. **Preprocessing / cleaning / labelling.** What was done, and is the raw version still available?
5. **Uses.** What has it been used for; what should it **not** be used for?
6. **Distribution.** Who gets it, under what licence, with what restrictions?
7. **Maintenance.** Who maintains it, how are errors fixed, is there versioning, is there a retention limit?

The questions are deliberately uncomfortable. “We do not know” is a valid answer — and is itself the finding.

Where these live. Hugging Face model and dataset cards are exactly this format. Keep the card in the *same repository as the training code*, generate its quantitative table *from the evaluation script*, and make it a merge requirement — a hand-copied table is stale within a week.

Exercise (15 minutes)

Take the last model you trained. Write its **intended use**, its **out-of-scope uses**, and one **quantitative table with slices**. Most people discover they cannot fill in the second one.

T. Gebru, J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. Daumé III and K. Crawford, "Datasheets for Datasets," *Communications of the ACM* 64(12):86–92, 2021, [arXiv:1803.09010v8](https://arxiv.org/abs/1803.09010v8).

Review before release

- ▶ A named reviewer who is **not** the model's author signs off the card and the disaggregated results.
- ▶ Scale the review to the stakes: a recommender for internal search does not need a committee; a credit model does.
- ▶ Record the decision *and the trade-off accepted*, so it can be revisited.

Red-teaming

- ▶ Deliberately try to make the system fail for a specific group or input type, before users do.
- ▶ Give the red team the failure taxonomy from this lecture as a starting checklist.

After release

- ▶ **Incident reporting.** A defined path for “the model did something harmful,” with an owner and a deadline — not a support ticket queue.
- ▶ **Contestability.** A person affected by a decision can ask for it to be reviewed by a human, and the request is logged.
- ▶ **Scheduled re-audit.** Put the date in the model card and treat it like a certificate expiry.
- ▶ **A kill switch.** Know in advance how you turn the model off and what the fallback is.

The common failure is not the absence of a policy. It is a policy with no owner and no date.

SDAIA AI Ethics Principles (2023)

Seven principles: fairness; privacy and security; humanity; social and environmental benefits; reliability and safety; transparency and explainability; accountability and responsibility.

Four risk tiers:

- ▶ **Little or no risk:** the principles are recommended.
- ▶ **Limited or high risk:** the principles are required, with additional controls for high risk.
- ▶ **Unacceptable risk:** prohibited, for example threats to the safety of life or the exploitation of children.

Applies to developers, designers and users of AI systems, and to the people they affect.

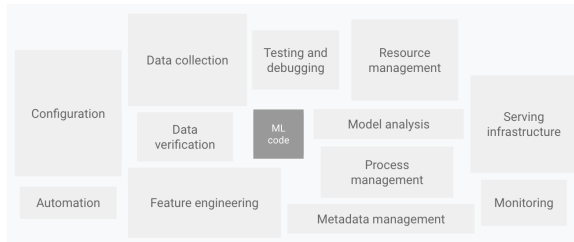
Personal Data Protection Law (PDPL)

- ▶ In force since 14 September 2023; enforced by SDAIA since 14 September 2024.
- ▶ Training sets, prediction logs and monitoring data are **personal data** whenever they relate to identifiable people: they need a lawful basis, a stated purpose, and no more data than that purpose requires.
- ▶ Breaches are reported to SDAIA within **72 hours**; data leaves the Kingdom only under its transfer rules.
- ▶ Fines up to **SAR 5 million** per violation, doubled for repeat offences.

*This lecture's tools map onto them: a disaggregated audit is evidence for **fairness**; a model card for **transparency** and **accountability**; and the prediction-log retention decision in the MLOps lecture later today is a **PDPL** decision.*

	What it is	When it matters to you
EU AI Act Regulation 2024/1689	(EU)	Binding EU law, with risk tiers close to SDAIA's: prohibited practices, high-risk systems, transparency duties, minimal risk
NIST AI RMF 1.0 (2023)	Voluntary US framework built on four functions: Govern, Map, Measure, Manage	Any system sold or used in the EU. Prohibitions apply since February 2025; after the 2026 Digital Omnibus, high-risk duties start in December 2027 (August 2028 for AI inside regulated products)
ISO/IEC 42001:2023	Certifiable management-system standard for AI	A ready-made structure for an internal AI risk process
		Showing a customer or regulator that the process exists and is audited

- ▶ **The common core is this lecture:** know the risk tier, test for disparate harm, document intended and out-of-scope use, keep a named human accountable, and monitor after release.
- ▶ **Dates move.** These are as of October 2026; check the current text before you rely on one.



Source: Google Cloud, “MLOps: Continuous delivery and automation pipelines in machine learning,” [Cloud Architecture Center](#),
Figure 1 — after Sculley et al., *Hidden Technical Debt in Machine Learning Systems*, NeurIPS 2015, Figure 1.

- ▶ Sculley et al. (2015): “*Only a small fraction of real-world ML systems is composed of the ML code . . . The required surrounding infrastructure is vast and complex.*”
- ▶ The grey box is what this course has been about. Everything around it is what makes it *useful*.

None of these are modelling problems. All of them are engineering problems.

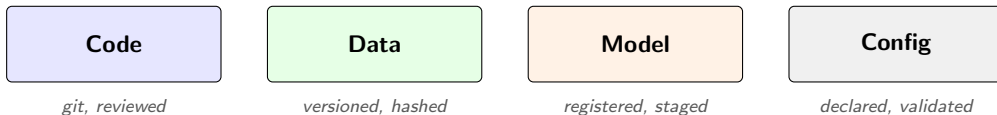
Symptom	Actual cause
Accuracy fell off a cliff overnight	An upstream team renamed a column; your feature is now all-null
Accuracy decayed slowly over a year	The world changed; your training distribution did not
"Works on my machine"	Unpinned dependency; a minor version bumped a default
Cannot reproduce last quarter's model	The training data was overwritten in place
Two teams report different numbers	Different splits, different seeds, different preprocessing
Nobody noticed for three months	Nothing was monitored except server uptime

See Paleyes, Urma & Lawrence, "Challenges in Deploying Machine Learning: a Survey of Case Studies," *ACM Computing Surveys*, 2022, [arXiv:2011.09926v3](https://arxiv.org/abs/2011.09926v3), for a catalogue of these from published industry post-mortems.

Working definition

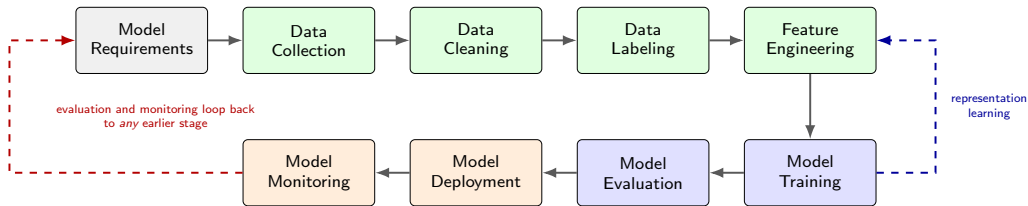
MLOps applies DevOps practices — version control, automated testing, continuous integration and delivery, monitoring — to machine learning systems, where the deliverable is not just code but **code + data + model** together.

A deployment is a *tuple* of all four — not a single commit.



- ▶ **Not** a product you buy. **Not** a job title you hire once. It is a set of habits plus a small amount of tooling.
- ▶ The habits matter more than the tools. A team with a spreadsheet and discipline beats a team with a platform and none.

The Lifecycle, Stage by Stage



Stages after Amershi et al., "Software Engineering for Machine Learning: A Case Study," ICSE-SEIP 2019, Figure 1. [Paper](#).

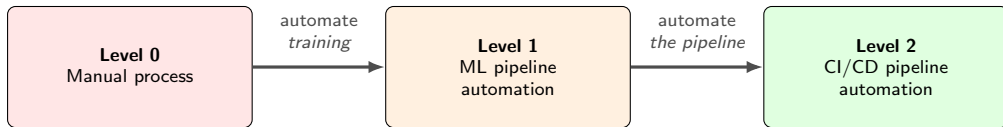
- ▶ **Green** is data work, **blue** is model work, **orange** is operations.
- ▶ The dashed arrows are why a linear project plan always slips.

Stage	Characteristic failure	Cheapest defence
Experiment	Results not attributable to any run	Experiment tracking
Validate	Leakage; tuned on the test set	Frozen holdout, nested CV
Package	Environment mismatch; unsafe artifact	Lockfile + container; safe format
Deploy	Training/serving skew in preprocessing	One shared transform, tested
Monitor	Silent degradation, nobody notices	Input and prediction distribution alerts
Retrain	Retrained on drifted, unvalidated data	Data validation before training

Training/serving skew

The single most common production bug in classical ML: the notebook computed a feature one way, the service computes it another way. The model is fine; the inputs are not.

Google's framing. Useful mostly because it names what you are allowed to not have yet.



- ▶ **Level 0** deploys a *model*. **Level 1** deploys a *training pipeline*. **Level 2** deploys *changes to that pipeline*, automatically and tested.
- ▶ Most organisations that believe they are at level 2 are at level 0 with a scheduled job.
- ▶ **Maturity is a cost, not a virtue.** Buy the level your problem justifies: how often must the answer change, what does a wrong prediction cost, and how many people must be able to reproduce the result?

Source: Google Cloud, "MLOps: Continuous delivery and automation pipelines in machine learning," [Cloud Architecture Center](#), 2020.

1. **Random number generators.** Weight initialisation, shuffling, dropout, data augmentation, train/test splits, bootstrap sampling in random forests.
2. **Parallelism.** Floating-point addition is not associative; sum the same numbers in a different order on a different thread and you get a different last bit — which compounds.
3. **Library and hardware versions.** A minor release changes a default; a different GPU picks a different convolution kernel.
4. **The data itself.** A table that is updated in place has no version. “The training set” is not a thing unless you make it one.
5. **Configuration.** The flag you changed by hand at 11pm and did not commit.

Order of attack

Seeds and data versioning buy you the most for the least effort. Bit-exact determinism on GPU is the most expensive and usually the least valuable.

Configuration as code

- ▶ Hyperparameters and paths belong in a *declared* config file (YAML, or a framework such as Hydra), not in argparse defaults scattered across three scripts.
- ▶ The config is committed, diffable and reviewable. “What changed between run 41 and run 42?” becomes a diff.
- ▶ Validate the config on load — a typo in a key name should crash, not silently fall back to a default.

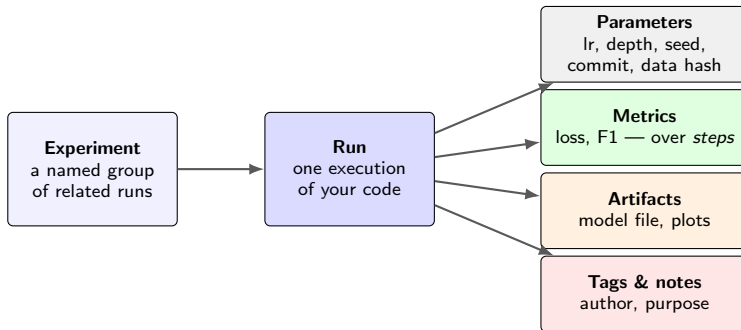
Log with every run

- ▶ git commit hash **and** whether the tree was dirty
- ▶ the full resolved config
- ▶ random seed(s)
- ▶ dataset identifier or content hash
- ▶ lockfile hash / image digest
- ▶ metrics, and the evaluation code version

The community version of this list is the *ML Reproducibility Checklist*: Pineau et al., “Improving Reproducibility in Machine Learning Research (A Report from the NeurIPS 2019 Reproducibility Program),” 2020, [arXiv:2003.12206v4](https://arxiv.org/abs/2003.12206v4).

Good news: once you adopt experiment tracking — the next section — five of these six items are logged for you automatically.

The Data Model: What a “Run” Contains



- ▶ **A run is append-only by convention.** You add to it, you do not edit it — so the record cannot drift away from what actually happened.
- ▶ Log metrics **per step**, not only at the end: a learning curve diagnoses ten times more than a single number.
- ▶ Log the **inputs** (commit, config, data hash) as parameters. Without them a run is a number without provenance.

Component	What it is
Tracking	An API and UI for logging parameters, code versions, metrics and artifacts when you run ML code, and for visualising the results afterwards.
Projects	A standard format for packaging and sharing reproducible data-science code, with its entry points and environment declared, so someone else can rerun it.
Models	A standard packaging format with “flavours”. Anything supporting the <code>python_function</code> flavour can be served as a REST endpoint, batch-scored in Spark, or exported to a cloud serving platform.
Model Registry	A centralised model store with versioning, aliases, tags and annotations, plus lineage back to the run that produced each version.

- ▶ They are designed to work together but each one is usable alone. **Start with Tracking** — it is three lines of code and it is where all the value is at first.
- ▶ MLflow is open source and runs entirely on your laptop: no account, no network, no cost.

Source: MLflow documentation, mlflow.org/docs/latest/ml. See also Zaharia et al., “Accelerating the Machine Learning Lifecycle with MLflow,” *IEEE Data Engineering Bulletin* 41(4), 2018.

```
import mlflow, mlflow.sklearn

mlflow.set_experiment("churn-baseline")

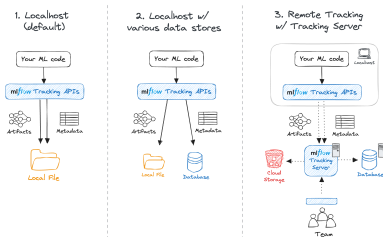
with mlflow.start_run(run_name="rf-depth-sweep"):
    mlflow.log_params({"n_estimators": 500, "max_depth": 12, "seed": 42})
    mlflow.log_param("data_version", "2026-03-14")

    model = RandomForestClassifier(n_estimators=500, max_depth=12,
                                   random_state=42).fit(X_tr, y_tr)

    proba = model.predict_proba(X_va)[:, 1]
    mlflow.log_metric("val_f1", f1_score(y_va, model.predict(X_va)))
    mlflow.log_metric("val_auc", roc_auc_score(y_va, proba))
    mlflow.log_artifact("confusion_matrix.png")
    mlflow.sklearn.log_model(model, name="model")
```

- ▶ `mlflow ui` then serves everything on `localhost:5000`. Nothing else to install.
- ▶ **Autologging** (`mlflow.autolog()`) captures parameters, metrics and the model for many frameworks with one line — useful, but log the *data version* and *seed* yourself; no autologger can guess those.

Where Does the Tracking Data Actually Live?



Source: MLflow documentation, “MLflow Tracking,” mlflow.org/docs/latest/ml/tracking.

- **Metadata** (params, metrics, tags) goes to a database; **artifacts** (model files, plots) go to object storage. They are separate stores for a reason: one is small and queried, the other is large and streamed.
- Solo work: local files, zero setup. Team work: one shared tracking server, so “my numbers” and “your numbers” live in the same place.

- ▶ **Grid search** tries every combination of a few values per hyperparameter. **Random search** samples combinations instead, and at the same budget it usually wins: only a few hyperparameters matter, and random search tries more values of each (Bergstra & Bengio, 2012).
- ▶ Both are **memoryless**: trial 90 learns nothing from trials 1–89, and an obviously bad trial still runs to the end.
- ▶ **Model-based (Bayesian) search** fixes the first: a cheap model of “settings $\rightarrow$ score” proposes where to try next. **Optuna**’s default sampler, TPE, picks settings that look like the best trials so far.
- ▶ **Pruning bad trials** fixes the second: stop a trial early when it lags the others at the same step. In the Optuna paper’s experiments, pruning was the bigger win.
- ▶ **Every trial is a run**: log it to the tracker, so the search can be compared, reproduced and audited. Hyperparameter optimisation (HPO) is experiment tracking on autopilot.

Designing the space

- ▶ **Few parameters, wide ranges** beats many parameters with narrow ones.
- ▶ Log scale for anything multiplicative.
- ▶ Bound the space with physics and budget, not superstition: if a depth of 40 will not fit in memory, do not offer it.
- ▶ If the best value sits on a **boundary**, your range was wrong — widen it and rerun.
- ▶ Tune the *pipeline*, not only the model: imputation strategy and scaler often matter more than `n_estimators`.

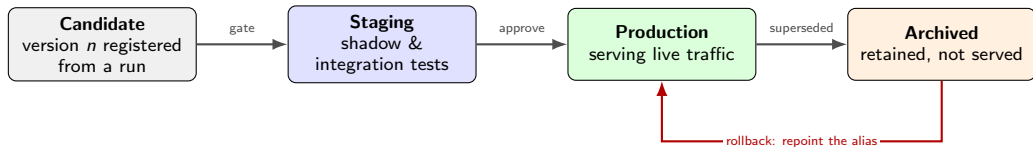
When to skip HPO entirely

- ▶ **The baseline is not built yet.** Tune nothing until a dumb model is logged.
- ▶ **The validation set is small.** With 200 samples, 500 trials just find the configuration that best overfits your split.
- ▶ **The gain is not decision-relevant.** +0.3 F1 points that change no business action is compute set on fire.
- ▶ **The data is the bottleneck.** A week of labelling usually beats a week of tuning.

Guard the test set

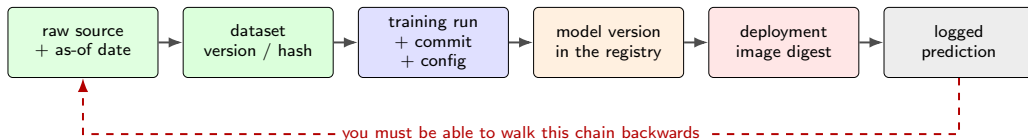
The HPO objective must be computed on **validation** data — ideally cross-validated. A search of 500 trials against a single small split is a very efficient overfitting machine.

The Model Registry: A Lifecycle, Not a Folder



- ▶ A registry stores **named models** with **numbered versions**, each carrying tags, metrics and a pointer back to the run that produced it.
- ▶ Deployments should reference a **mutable alias** (“champion”, “challenger”) that points at an **immutable version**. Promotion is then a metadata change, not a redeploy — and rollback is repointing the alias.
- ▶ MLflow implements exactly this with registered models, versions, aliases and tags. (Older material describes fixed “stages”; aliases and tags are the current mechanism.)
- ▶ **Never delete the previous production version.** It is your rollback target and your audit record.

Lineage: Answering “Where Did This Number Come From?”



- ▶ The chain is only as strong as its weakest link. One un-versioned step — “we pulled the customers table” — breaks the whole trace.
- ▶ **Two questions a customer, a reviewer or your own on-call will ask:** which model produced *this* decision, and what data was it trained on? Both must be answerable in minutes, from a log, without a person’s memory.
- ▶ Nothing exotic is required: an ID at each hop, stored with the next hop. Experiment tracking plus a registry plus prediction logs gives you the whole chain.

What happened

1. Tuesday 09:14 — the manual-review queue triples. No alert fires; the service is healthy.
2. 11:40 — an analyst notices.
Predicted-positive rate went from 4% to 19% overnight.
3. 12:20 — the prediction log shows `tenure_months` is 0 for 80% of requests since 02:00.
4. 12:35 — an upstream job began writing nulls; the serving code imputed them to zero, silently.

What made it survivable

- ▶ **Prediction logs** contained the post-preprocessing features — so the cause was found in minutes, not weeks.
- ▶ **Model version** was on every record, ruling the model out immediately.
- ▶ **The previous version was still in the registry**, so a rollback was one alias change.

What was missing: a null-rate check on serving inputs, and an alert on prediction distribution. Both were one-day fixes.

The lesson

The model was never wrong. **Rolling back the model would not have fixed anything** — which is exactly why you need the lineage to tell you where to look.

Ship documentation as an artifact

- ▶ A **model card** — intended use, training data, disaggregated performance, known limitations, owner — registered alongside the model version, not written in a wiki that rots.
- ▶ A **datasheet** — the same discipline for the dataset it was trained on.
- ▶ Generate what you can **automatically** from the tracking run: metrics, data version, config, commit. Only the judgement calls need a human.

The content and ethics of these documents were covered in depth earlier in the course; here we only care that the pipeline *produces and versions* them.

A minimal governance checklist

- ▶ Every production model has a named **owner** and a documented purpose.
- ▶ Every model has a **review date**; unreviewed models get retired.
- ▶ Decisions affecting people are **logged and explainable**.
- ▶ There is a written **escalation path** for a contested prediction.
- ▶ Retraining and promotion require a recorded **approval**.

Mitchell et al., “Model Cards for Model Reporting,” FAT* 2019, [arXiv:1810.03993v2](#); Gebru et al., “Datasheets for Datasets,” *Communications of the ACM* 64(12), 2021, [arXiv:1803.09010v8](#).

Hand-off: bias auditing, fairness definitions and interpretability belong to the responsible-AI material seen earlier. MLOps supplies the *mechanisms* — lineage, prediction logs, versioned artifacts — that make those commitments enforceable rather than aspirational.

“Ship the Model” — Ship *What*, Exactly?

Learned parameters
weights, splits,
support vectors

Preprocessing
the *fitted* scaler,
encoder, imputer

Inference code
predict, threshold,
post-processing

Environment
library versions,
runtime

Metadata
schema, metrics,
lineage, owner

- ▶ Forgetting the **fitted** preprocessing is the classic production bug: a scaler refitted on the serving batch silently rescales everything and the model quietly degrades.
- ▶ Forgetting the **input schema** means you discover a renamed column at 3am rather than at deploy time.
- ▶ Forgetting the **owner** means nobody is paged when it breaks.

Rule

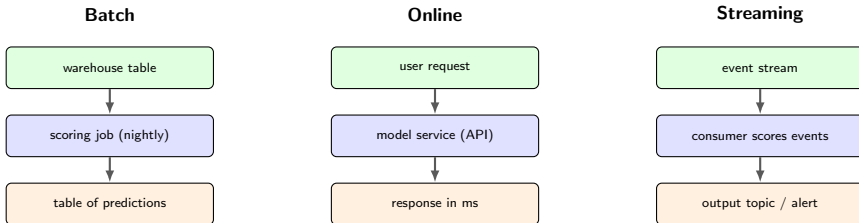
Serialise the **whole pipeline** — `Pipeline([("scale", ...), ("clf", ...)])` — never the estimator alone. One artifact, one predict, no room for skew.

Format	Stores	Good	Bad
<code>pickle</code> / <code>joblib</code>	Arbitrary Python objects	Works for anything; zero friction	Executes code on load ; breaks across library versions
<code>skops</code>	scikit-learn estimators	Contents inspectable before loading; safer than pickle	scikit-learn ecosystem only
<code>safetensors</code>	Tensors + a JSON header	No code execution; fast zero-copy load	Tensors only — no pipeline logic
ONNX	A computation graph	Serve without Python; portable across runtimes	Not every estimator or op converts
Framework native	<code>.pt</code> , <code>SavedModel</code> , <code>.ubj</code>	Full fidelity; official support	Ties you to that framework's version

The pickle warning, in scikit-learn's own words

“pickle (and joblib and cloudpickle by extension) has many documented security vulnerabilities by design and should only be used if the artifact ... is coming from a trusted and verified source. You should never load a pickle file from an untrusted source, similarly to how you should never execute code from an untrusted source.”

Source: scikit-learn documentation, “Model persistence,” scikit-learn.org/stable/model_persistence.html.



	Batch	Online	Streaming
Latency	Hours	Milliseconds	Seconds
Cost	Lowest — one big job	Highest — always-on capacity	In between
Failure	Rerun the job	Users see errors	Backlog grows
Use when	The input exists before the question	The input arrives with the question	Continuous events, bounded delay

Default to batch. If a nightly table of predictions solves the problem, build that: an order of magnitude cheaper to run and to operate, and it fails without waking anyone.

REST/JSON vs gRPC

- ▶ **REST + JSON:** human-readable, debuggable with `curl`, works with every client. Verbose; parsing costs real time on large payloads.
- ▶ **gRPC + protobuf:** compact binary, schema-checked, streaming, faster for high-volume internal traffic. Harder to inspect.
- ▶ **Pick REST first.** Move to gRPC when profiling says serialisation is your bottleneck — rarely true for a scikit-learn model.

Budget the whole request

Network in/out	5–20 ms
Feature lookup (DB / feature store)	5–50 ms
Preprocessing	1–5 ms
Model forward pass	1–10 ms
Post-processing, logging	1–5 ms

The model is often the *smallest* term. Optimise what you measured, not what you assumed.

Quote p95 and p99, never the mean. The tail is what users experience and what times out upstream.

Throughput vs latency

Batching requests on the server raises throughput and *raises* latency. Pick which one the product actually needs, then set the batch window accordingly.

	CPU serving	GPU serving
Fits	Trees, linear models, small nets, most tabular ML	Large nets, transformers, image and audio models
Cost profile	Cheap, scales horizontally, scale-to-zero is easy	Expensive per hour; idle time is pure waste
Wins when	Requests are small and sporadic	Requests are large or can be <i>batched</i>
Watch out for	Thread oversubscription inside the container	Low utilisation — a GPU at 5% is a very costly CPU

- ▶ **For the classical ML in this course, CPU is almost always right.** A gradient-boosted tree does not benefit from a GPU at inference.
- ▶ Before buying a GPU, try: fewer features, a smaller model, ONNX Runtime, or simply caching repeated inputs. These are usually larger wins than new hardware.
- ▶ *Next:* when the model itself is the bottleneck — on a device, or a large network at high volume — make it smaller: quantisation, pruning and distillation.

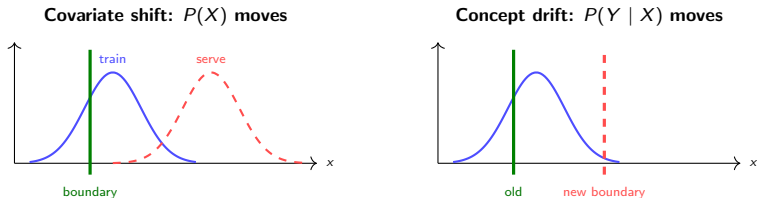
Four Ways a Deployed Model Goes Wrong

What	Looks like	Timescale
Upstream breakage	A feature becomes all-null, all-zero, or wrongly typed	Instant, catastrophic
Data drift	$P(X)$ moves: new customers, new devices, new regions	Weeks to months
Concept drift	$P(Y X)$ moves: the same inputs now imply a different answer	Days to years
Model staleness	Nothing drifted much, but the model has not seen a year of new data	Slow and boring

- ▶ **Upstream breakage is the most common and the most fixable.** It is a data engineering bug, and a schema check catches it in minutes rather than months.
- ▶ The other three cannot be prevented — only detected and answered with a retrain.
- ▶ **Server uptime tells you none of this.** A model service returning 200s at 8 ms with catastrophically wrong predictions is a perfectly healthy service.

The model learned a joint distribution

Training gave you $P_{\text{train}}(X, Y) = P(X)P(Y | X)$. Deployment assumes $P_{\text{serve}} = P_{\text{train}}$. Drift is any way that assumption fails.



- ▶ **Left:** the inputs moved but the rule is unchanged. Your model may still be right — it is just being asked questions it was not trained on.
- ▶ **Right:** the inputs look identical and the *answer* changed. Nothing in the input distribution will tell you. Only labels will.
- ▶ **This is why input monitoring alone is not enough** — and why it is still worth having, because it is the only signal available immediately.

What you can measure today

- ▶ **Schema and range checks** — type, nullability, allowed categories, min/max. Cheap, and catches the common case.
- ▶ **Per-feature distribution distance** against a fixed reference window: Kolmogorov–Smirnov for continuous, chi-square or PSI for categorical.
- ▶ **Prediction distribution.** If your model predicted “churn” for 4% of users all year and today it says 19%, something changed — even if you cannot yet say whether it was right.
- ▶ **Confidence / entropy** of predictions, and the rate of out-of-range inputs.

Population Stability Index

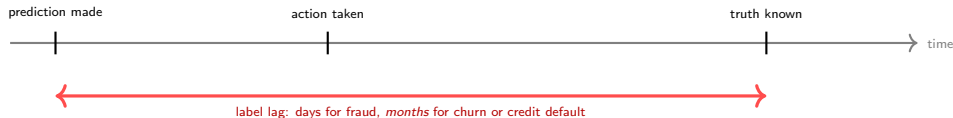
$$\text{PSI} = \sum_b (p_b - q_b) \ln \frac{p_b}{q_b}$$

over bins b of a feature; p is the reference, q the current window.

Credit-scoring rule of thumb: < 0.1 stable, 0.1 – 0.25 moderate shift, > 0.25 significant. A convention, not a theorem — calibrate it on your own history.

The large- n trap

With a million rows per day, every statistical test rejects. Alert on **effect size** and on a **trend**, not on a p -value.



- ▶ You cannot compute accuracy in production until the labels arrive. For a 12-month loan default model, that is a **12-month** feedback delay.
- ▶ **Proxies to use in the meantime:**
 - input and prediction distribution monitoring (previous slide);
 - **business metrics** that respond faster — click-through, override rate, complaint volume, manual-review queue length;
 - **human spot-checks:** label a small random sample every week, deliberately. A hundred labels a week is a real accuracy estimate.
- ▶ **Beware biased feedback:** if you only ever see the outcome for applicants you approved, your production “accuracy” is measured on a censored sample.

Alerting rules that survive contact

- ▶ Every alert names an **owner** and a **first action**. An alert with neither will be muted within a month.
- ▶ Page on **breakage**; e-mail a weekly digest for **drift**. They are different urgencies.
- ▶ Alert on a **sustained** threshold crossing (e.g. 3 days), not on one noisy window.
- ▶ Track how often each alert was actionable. Delete the ones that never were.

When to retrain

- ▶ **Scheduled** — weekly or monthly. Boring, predictable, and correct far more often than people expect. Start here.
- ▶ **Volume-based** — every N new labelled examples.
- ▶ **Performance-based** — metric falls below a floor. Requires labels.
- ▶ **Drift-based** — input distance exceeds a threshold. Available immediately, but noisier.

Automatic retraining needs a gate

A retrain that ships without a **validation gate** is an automated way to deploy a worse model. The new candidate must beat the incumbent on a held-out set before it is promoted — and the pipeline must be able to refuse.

Record, per request

- ▶ A request ID and a timestamp
- ▶ The **input features as the model saw them** (post-preprocessing)
- ▶ The raw output and the final decision
- ▶ The **model version** and config version
- ▶ Latency, and any fallback that fired

Join the label in later, on the request ID. That join is what makes every metric on the previous slides computable at all.

What this buys you

- ▶ **Debugging:** replay the exact input that produced the complaint.
- ▶ **Training data:** today's production log is next quarter's training set.
- ▶ **Audit:** answer "why was this person declined, by which model, on what inputs?"

Privacy is a design input

Prediction logs contain personal data. Decide retention, access and pseudonymisation *before* you switch logging on, not after.

Two layers

- ▶ **Schema:** columns present, types correct, categories in the allowed set, nullability as declared. This is a hard failure — stop the pipeline.
- ▶ **Quality / statistics:** distributions within expected bounds, null rate below a ceiling, no duplicate keys, freshness within an SLA. This is usually a warning, and a gate on retraining.

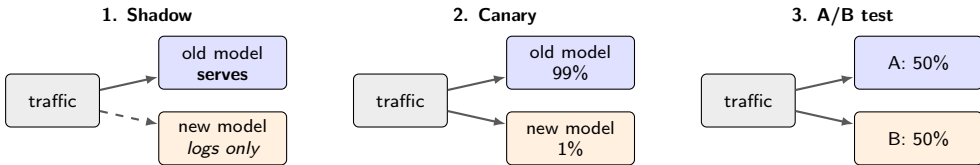
Write the schema once and reuse it at three points: training input, retraining input, and serving input. That single artifact is your best defence against training/serving skew.

```
def validate(df: pd.DataFrame) -> None:
    assert set(SCHEMA) <= set(df.columns), "missing column"
    assert df["age"].between(18, 120).all()
    assert df["contract"].isin(CONTRACTS).all()
    assert df["monthly_charges"].isna().mean() < 0.01
    assert df["id"].is_unique
    assert (NOW - df["event_ts"].max()).days < 2 # freshness
```

Tools that formalise this: Great Expectations, Pandera, TensorFlow Data Validation, dbt tests. Twenty lines of assert statements already capture most of the value.

Test type	What it asserts
Smoke / shape	The model loads, accepts one row, returns a probability in $[0, 1]$. Catches most deployment breakage.
Minimum functionality	Cases so easy that failure is unambiguous: a customer who cancelled last month must score high risk.
Invariance	Perturbations that must <i>not</i> change the prediction: reordering columns, changing a customer ID, whitespace in a category.
Directional	Perturbations with a known sign: raising the price should not <i>lower</i> predicted churn.
Golden set	A fixed, curated set of examples with expected outputs, reviewed by a human and version-controlled.
Regression / slice	Overall metric above a floor <i>and</i> no important segment worse than the incumbent by more than δ .

- ▶ **Slice tests matter more than the headline metric.** An overall $+1\%$ that hides -8% for one region is a regression, not an improvement.
- ▶ The invariance/directional/minimum-functionality vocabulary is from Ribeiro et al., “Beyond Accuracy: Behavioral Testing of NLP Models with CheckList,” ACL 2020, [arXiv:2005.04118v1](https://arxiv.org/abs/2005.04118v1) — the idea transfers directly to tabular models.



- ▶ **Shadow:** the new model sees real traffic and its predictions are logged and compared, but never returned. Catches crashes, latency blow-ups and wild disagreement at **zero user risk**. Do this first, always.
- ▶ **Canary:** a small slice of real traffic gets the new model. Watch error rate, latency and the business metric; ramp 1% → 10% → 50%, with an automatic rollback rule written down *before* you start.
- ▶ **A/B test:** a designed experiment to measure whether the new model actually improves the *business* outcome — which is not the same question as whether its F1 is higher.
- ▶ **Rollback must be one command and it must be rehearsed.** The previous image digest and the previous model version are both still in the registry: that is what they are for.

- ▶ The latency budget showed that for a typical web service the model is the *smallest* term. That flips when the model runs **on the device** (a camera, a phone, a vehicle), when it is a **large network** served at high volume, or when **memory or energy** is the constraint.
- ▶ Then you make the model itself cheaper. Three levers, often combined:

	What changes	What you gain	Retraining?
Quantisation	Fewer bits per weight and activation (FP32 $\rightarrow$ INT8)	4 $\times$ smaller weights; faster where INT8 kernels exist	None, or a fine-tune (QAT)
Pruning	Removes weights, channels or layers	Smaller; faster only if the pruning is <i>structured</i>	Usually a fine-tune
Distillation	Trains a small <i>student</i> to mimic a large <i>teacher</i>	Any smaller architecture you choose	A full training run

Rule

Measure on the **target hardware**, before and after. A compression that does not move p95 latency, memory or energy on the device is cost without benefit.

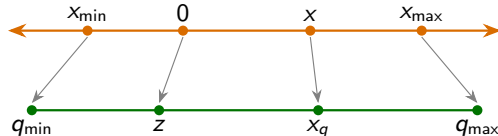
- ▶ Map each real value x to an integer x_q with a **scale** s and a **zero-point** z :

$$x_q = \text{clip}(\text{round}(x/s + z), q_{\min}, q_{\max})$$

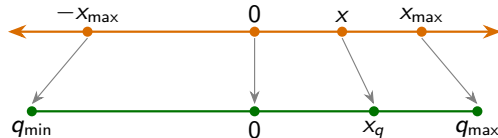
and dequantising gives back $x \approx s(x_q - z)$.

- ▶ **INT8** has 256 levels, so s comes from the observed range of x . **Symmetric** quantisation fixes $z = 0$.
- ▶ **Rounding and clipping lose information**, and dequantising does not get it back. One outlier stretches the range and wastes levels, which is why scales are often kept **per channel** rather than per tensor.

Affine (asymmetric)



Symmetric



Real values (top) to integer levels (bottom). After Wang & Spindler, NVIDIA Technical Blog (2025), Figure 2.

Method	How it works	Use it when
Post-training, dynamic	Weights are quantised ahead of time; activation scales are computed on the fly for each input	A first try: needs no data. Linear- and LSTM-heavy models on CPU
Post-training, static	A calibration set of representative inputs fixes the activation scales once	Fastest at run time; CNNs and edge devices
Quantisation-aware training	Training simulates the rounding (<i>fake quantisation</i> , with gradients passed straight through it), so the weights adapt to it	Post-training loses too much accuracy, e.g. at 4 bits or on small models

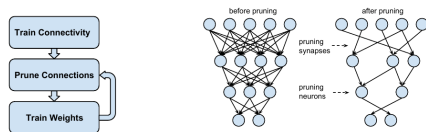
- ▶ **The calibration set is training data for the quantiser.** Version it with the model, and check it still looks like production when the inputs drift.
- ▶ **Tools:** **torchao** for PyTorch (`torch.ao.quantization` is deprecated), **ONNX Runtime** quantisation for exported models, **TensorRT** or **OpenVINO** for NVIDIA or Intel targets.

- ▶ **Fake quantisation** rounds the weights to the integer grid and straight back to floats. The model still computes in FP32, so it is no smaller and no faster. It is how you *measure the accuracy cost* before deploying anything, and how quantisation-aware training works.
- ▶ **Real gains need a runtime with integer kernels:** ONNX Runtime on CPUs, TensorRT on NVIDIA GPUs, Core ML on Apple devices, OpenVINO on Intel. You export the model (for example to ONNX), quantise it with a calibration set, and run it there.
- ▶ **How much faster depends on the chip.** CPUs with INT8 dot-product instructions gain the most. Benchmark on the device you will ship to.
- ▶ **Not every layer should be quantised.** A tensor that mixes values of very different sizes loses the small ones to a single shared scale. Keep such layers in float, and re-measure accuracy after every change.

In Lab 1

You quantise a CNN by hand, then for real with ONNX Runtime, then a YOLO11 object detector, and compare the versions live on a webcam.

- ▶ Trained networks are heavily over-parameterised: many weights barely affect the output.
- ▶ **Magnitude pruning:** remove the smallest weights, fine-tune the survivors to recover accuracy, repeat. Han et al. (2015) cut AlexNet's weights $9\times$ and VGG-16's $13\times$ with no loss of accuracy.
- ▶ **Unstructured** pruning zeroes individual weights: high compression, but a sparse matrix runs no faster on standard kernels.
- ▶ **Structured** pruning removes whole channels, filters or layers: less compression, but a smaller *dense* model that is faster on any hardware.



Han et al., [arXiv:1506.02626](https://arxiv.org/abs/1506.02626), Figures 2–3.

```
import torch.nn.utils.prune as prune
# zero the 50% smallest weights
prune.l1_unstructured(layer, "weight",
                      amount=0.5)
prune.remove(layer, "weight")
```

- ▶ Train a large **teacher** for accuracy, then train a small **student** to reproduce the teacher's *output probabilities*, not just the labels (Hinton, Vinyals & Dean, 2015).
- ▶ A softmax with temperature T softens those probabilities:

$$q_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

- ▶ The soft targets carry what the teacher knows about **which wrong answers are nearly right**. A hard label throws that away.

	cow	car	dog	cat
Label	0	1	0	0
Teacher, $T=1$	10^{-6}	0.9	0.1	10^{-9}
Teacher, $T=5$	0.04	0.58	0.37	0.01

At $T=1$ cow and cat both look like zero; at $T=5$ the student can see that the teacher ranks cow well above cat. Adapted from Hinton, Vinyals & Dean (2015).

- ▶ The student minimises a mix of two cross-entropies:

$$L = \lambda \underbrace{\text{CE}(y, q_S^{(T=1)})}_{\text{match the labels}} + (1 - \lambda) T^2 \underbrace{\text{CE}(p_T^{(T)}, q_S^{(T)})}_{\text{match the teacher}}$$

- ▶ The soft term's gradients shrink as $1/T^2$, hence the T^2 : without it, raising T would quietly weaken the teacher's signal.
- ▶ **What it buys:** you choose the student's architecture, so it can be sized for the device. DistilBERT keeps 97% of BERT's language-understanding performance with 40% fewer parameters, and runs 60% faster (Sanh et al., 2019).
- ▶ **What it costs:** a full training run, and the teacher's training data. The student also inherits the teacher's mistakes, including its weak slices.

- ▶ A quantised, pruned or distilled model is a **new model version**. Track it as a run (method, bit-width, sparsity, calibration-set hash) and register it next to the original.
- ▶ **Re-run the full evaluation, per slice**, not just the headline metric. Compression tends to cost the rare cases most: the Responsible AI lecture's *learning bias*, now in deployment.
- ▶ **Benchmark on the target hardware** with production-shaped inputs: p95 latency, memory, energy. A laptop benchmark says little about an embedded camera.
- ▶ **Ship it like any release**: shadow it against the original, then canary it. The uncompressed version stays in the registry as the rollback target.

Order of effort

Post-training quantisation first (minutes, no retraining) → quantisation-aware training, or structured pruning plus a fine-tune → distillation into a smaller architecture. Stop as soon as the target is met. Han et al. (2016) combined pruning, quantisation and Huffman coding to shrink AlexNet 35× and VGG-16 49× with no loss of accuracy.

Lab 1	Quantisation, fewer bits per number. Quantise Day 2's Fashion-MNIST CNN by hand (symmetric vs affine, per tensor vs per channel, 8 bits down to 2), run it in INT8 with ONNX Runtime, then quantise a YOLO11 detector and find out what can go wrong.
Live demo	On your own laptop, <code>python yolo_live_demo.py</code> : YOLO11m at FP32 and INT8, the smaller YOLO11n, and FP16 on a GPU, switched live on your webcam.
Lab 2	Pruning and distillation, then ship one. Prune the CNN to 95% sparsity and recover it, distil it into a 4× smaller student from 1,000 labels, then gate every variant per class, on size and on speed, log them to MLflow and register the release.

Where to run them

On Colab or your own laptop's CPU; no GPU needed. The live demo needs your own laptop, because Colab cannot reach your webcam.

- ▶ **Bias enters at every stage**, from the data to the deployment loop. Dropping the sensitive column does not remove it; disaggregated evaluation finds it.
- ▶ **Fairness definitions conflict**. Choose one for the decision at hand, say why, and write it in the model card.
- ▶ **Explanations are models of the model**. Check them before trusting them: permutation importance over built-in importance, and sanity checks for heatmaps.
- ▶ **Governance is a process**: review before release, documentation, contestability and a re-audit date, within SDAIA's principles and the PDPL.
- ▶ **The ML code is the small box**. Reproducible runs, a tracker and a registry turn experiments into versions you can ship and roll back.
- ▶ **A deployed model decays**. Monitor its inputs and predictions before the labels arrive, and never flip the switch: shadow first, then canary.
- ▶ **Compression is a model change**. Quantise first, then prune or distil; re-measure per class, on the target hardware, and gate it like any release.

Variant	Size	Accuracy	Note
CNN, FP32	1.69 MB	89.1%	Lab 1's baseline
CNN, INT8 with ONNX Runtime	0.43 MB	89.1%	2.6× faster on a batch
YOLO11s, everything INT8	10.2 MB	mAP 0.000	one scale for pixels and probabilities
YOLO11s, INT8, head kept in FP32	10.2 MB	mAP 0.561	FP32: 38.1 MB, mAP 0.575
Teacher pruned 95% + fine-tune	1.69 MB, zipped 0.20 MB	89.9%	the zeros are still stored
Student alone, 1,000 labels	0.43 MB	82.3%	37.5% on shirts
Student distilled, INT8	0.12 MB	88.0%	shipped: 14× smaller than the teacher

Live: YOLO11m on the instructor's laptop

Frames processed per second: 16 in FP32 and 27 in INT8 on the CPU, 77 for the smaller YOLO11n, and 134 in FP16 on the laptop's GPU. Yours will differ, which is why you benchmark on the target hardware.

1. J. Angwin, J. Larson, S. Mattu and L. Kirchner, “Machine Bias,” *ProPublica*, 23 May 2016. [propublica.org](https://www.propublica.org/methodology/How-We-Analyzed-the-COMPAS-Recidivism-Algorithm) (methodology: “How We Analyzed the COMPAS Recidivism Algorithm”).
2. J. Dastin, “Amazon scraps secret AI recruiting tool that showed bias against women,” *Reuters*, 10 October 2018. (*News reporting, not a peer-reviewed study.*)
3. J. Buolamwini and T. Gebru, “Gender Shades: Intersectional Accuracy Disparities in Commercial Gender Classification,” *Proceedings of Machine Learning Research* 81:77–91 (FAT* 2018). proceedings.mlr.press
4. Z. Obermeyer, B. Powers, C. Vogeli and S. Mullainathan, “Dissecting racial bias in an algorithm used to manage the health of populations,” *Science* 366(6464):447–453, 2019. [doi:10.1126/science.aax2342](https://doi.org/10.1126/science.aax2342)
5. H. Suresh and J. V. Guttag, “A Framework for Understanding Sources of Harm throughout the Machine Learning Life Cycle,” EAAMO 2021. [arXiv:1901.10002v5](https://arxiv.org/abs/1901.10002v5)
6. T. Bolukbasi, K.-W. Chang, J. Zou, V. Saligrama and A. Kalai, “Man is to Computer Programmer as Woman is to Homemaker? Debiasing Word Embeddings,” *NeurIPS* 2016. [arXiv:1607.06520v1](https://arxiv.org/abs/1607.06520v1)
7. H. Gonen and Y. Goldberg, “Lipstick on a Pig: Debiasing Methods Cover up Systematic Gender Biases in Word Embeddings But do not Remove Them,” *NAACL* 2019. [arXiv:1903.03862v2](https://arxiv.org/abs/1903.03862v2)
8. C. Dwork, M. Hardt, T. Pitassi, O. Reingold and R. Zemel, “Fairness Through Awareness,” *ITCS* 2012. [arXiv:1104.3913v2](https://arxiv.org/abs/1104.3913v2)

9. M. Hardt, E. Price and N. Srebro, “Equality of Opportunity in Supervised Learning,” NeurIPS 2016. [arXiv:1610.02413v1](#)
10. J. Kleinberg, S. Mullainathan and M. Raghavan, “Inherent Trade-Offs in the Fair Determination of Risk Scores,” ITCS 2017. [arXiv:1609.05807v2](#)
11. A. Chouldechova, “Fair prediction with disparate impact: A study of bias in recidivism prediction instruments,” *Big Data* 5(2):153–163, 2017. [arXiv:1703.00056v1](#)
12. M. J. Kusner, J. R. Loftus, C. Russell and R. Silva, “Counterfactual Fairness,” NeurIPS 2017. [arXiv:1703.06856v3](#)
13. S. Barocas, M. Hardt and A. Narayanan, *Fairness and Machine Learning: Limitations and Opportunities*, MIT Press, 2023. [fairmlbook.org](#)
14. F. Kamiran and T. Calders, “Data preprocessing techniques for classification without discrimination,” *Knowledge and Information Systems* 33(1):1–33, 2012.
15. A. Agarwal, A. Beygelzimer, M. Dudík, J. Langford and H. Wallach, “A Reductions Approach to Fair Classification,” ICML 2018. [arXiv:1803.02453v3](#)
16. C. Rudin, “Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead,” *Nature Machine Intelligence* 1:206–215, 2019. [arXiv:1811.10154v3](#)
17. S. M. Lundberg and S.-I. Lee, “A Unified Approach to Interpreting Model Predictions,” NeurIPS 2017. [arXiv:1705.07874v2](#)

18. S. M. Lundberg et al., “From local explanations to global understanding with explainable AI for trees,” *Nature Machine Intelligence* 2:56–67, 2020. doi:10.1038/s42256-019-0138-9 (preprint: [arXiv:1905.04610v1](#), “Explainable AI for Trees”)
19. M. T. Ribeiro, S. Singh and C. Guestrin, “‘Why Should I Trust You?’: Explaining the Predictions of Any Classifier,” KDD 2016. [arXiv:1602.04938v3](#)
20. D. Slack, S. Hilgard, E. Jia, S. Singh and H. Lakkaraju, “Fooling LIME and SHAP: Adversarial Attacks on Post hoc Explanation Methods,” AIES 2020. [arXiv:1911.02508v2](#)
21. I. E. Kumar, S. Venkatasubramanian, C. Scheidegger and S. Friedler, “Problems with Shapley-value-based explanations as feature importance measures,” ICML 2020. [arXiv:2002.11097v2](#)
22. K. Simonyan, A. Vedaldi and A. Zisserman, “Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps,” ICLR 2014 Workshop. [arXiv:1312.6034v2](#)
23. M. Sundararajan, A. Taly and Q. Yan, “Axiomatic Attribution for Deep Networks,” ICML 2017. [arXiv:1703.01365v2](#)
24. R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh and D. Batra, “Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization,” ICCV 2017. [arXiv:1610.02391v3](#)
25. J. Adebayo, J. Gilmer, M. Muelly, I. Goodfellow, M. Hardt and B. Kim, “Sanity Checks for Saliency Maps,” NeurIPS 2018. [arXiv:1810.03292v3](#)
26. S. Jain and B. C. Wallace, “Attention is not Explanation,” NAACL 2019. [arXiv:1902.10186v3](#)

27. S. Wiegreffe and Y. Pinter, “Attention is not not Explanation,” EMNLP 2019. [arXiv:1908.04626v2](#)
28. M. Mitchell, S. Wu, A. Zaldivar, P. Barnes, L. Vasserman, B. Hutchinson, E. Spitzer, I. D. Raji and T. Gebru, “Model Cards for Model Reporting,” FAT* 2019. [arXiv:1810.03993v2](#)
29. T. Gebru, J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. Daumé III and K. Crawford, “Datasheets for Datasets,” *Communications of the ACM* 64(12):86–92, 2021. [arXiv:1803.09010v8](#)
30. C. Molnar, *Interpretable Machine Learning*, 2nd ed., 2022. [christophm.github.io](#)
31. Software used for the figures in this deck: [shap](#), [lime](#), [pytorch-grad-cam](#), [fairlearn](#).
32. Saudi Data and Artificial Intelligence Authority (SDAIA), *AI Ethics Principles*, 2023. [sdaia.gov.sa](#)
33. Kingdom of Saudi Arabia, *Personal Data Protection Law* and its Implementing Regulations, in force 14 September 2023. [sdaia.gov.sa](#)
34. Regulation (EU) 2024/1689 (the AI Act), as amended by Regulation (EU) 2026/1744 (the Digital Omnibus on AI). [eur-lex.europa.eu](#)
35. NIST, *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*, NIST AI 100-1, January 2023. [doi:10.6028/NIST.AI.100-1](#)
36. ISO/IEC 42001:2023, *Information technology — Artificial intelligence — Management system*.

1. Sculley, Holt, Golovin, Davydov, Phillips, Ebner, Chaudhary, Young, Crespo & Dennison, “Hidden Technical Debt in Machine Learning Systems,” *Advances in Neural Information Processing Systems 28* (NIPS 2015). [Paper](#) (Figure 1 — the “small box”).
2. Sculley et al., “Machine Learning: The High-Interest Credit Card of Technical Debt,” SE4ML Workshop, NIPS 2014. [Paper](#) (the earlier, longer version of the same argument).
3. Breck, Cai, Nielsen, Salib & Sculley, “The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction,” *Proc. IEEE Big Data*, 2017. [Paper](#).
4. Google Cloud, “MLOps: Continuous delivery and automation pipelines in machine learning,” Cloud Architecture Center, 2020. [Article](#) (maturity levels 0/1/2; source of the “elements of ML” figure).
5. Amershi, Begel, Bird, DeLine, Gall, Kamar, Nagappan, Nushi & Zimmermann, “Software Engineering for Machine Learning: A Case Study,” ICSE-SEIP 2019. [Paper](#) (the nine-stage workflow).
6. Paleyes, Urma & Lawrence, “Challenges in Deploying Machine Learning: a Survey of Case Studies,” *ACM Computing Surveys*, 2022. [arXiv:2011.09926v3](#).
7. Zinkevich, “Rules of Machine Learning: Best Practices for ML Engineering,” Google. [Guide](#) (43 rules; the best single practitioner document on this topic).
8. Zaharia, Chen, Davidson, Ghodsi, Hong, Konwinski, Murching, Nykodym, Ogilvie, Parkhe, Xie & Zumar, “Accelerating the Machine Learning Lifecycle with MLflow,” *IEEE Data Engineering Bulletin* 41(4), 2018, pp. 39–45. [PDF](#).

9. MLflow documentation — “MLflow Tracking,” “MLflow Projects,” “MLflow Models” and “Model Registry.” mlflow.org/docs/latest/ml (source of the tracking-setup and tracking-UI figures).
10. Weights & Biases documentation and pricing. [Docs](#), [Pricing](#) (accessed 2026).
11. Bergstra, Bardenet, Bengio & Kégl, “Algorithms for Hyper-Parameter Optimization,” NIPS 2011. [Paper](#) (the original TPE).
12. Bergstra & Bengio, “Random Search for Hyper-Parameter Optimization,” *JMLR* 13, 2012, pp. 281–305. [Paper](#).
13. Akiba, Sano, Yanase, Ohta & Koyama, “Optuna: A Next-generation Hyperparameter Optimization Framework,” KDD 2019. [arXiv:1907.10902v1](#) (source of the pruning figure, Figure 11(a)).
14. Optuna documentation — [TPESampler](#), [SuccessiveHalvingPruner](#), [create_study](#).
15. Li, Jamieson, DeSalvo, Rostamizadeh & Talwalkar, “Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization,” *JMLR* 18, 2018. [arXiv:1603.06560v4](#).
16. Li et al., “A System for Massively Parallel Hyperparameter Tuning,” MLSys 2020. [arXiv:1810.05934v5](#) (ASHA).
17. Pineau, Vincent-Lamarre, Sinha, Larivière, Beygelzimer, d’Alché-Buc, Fox & Larochelle, “Improving Reproducibility in Machine Learning Research (A Report from the NeurIPS 2019 Reproducibility Program),” *JMLR* 22(164), 2021. [arXiv:2003.12206v4](#) (the ML Reproducibility Checklist).
18. PyTorch documentation, “Reproducibility.” docs.pytorch.org.

19. scikit-learn documentation, “Model persistence.” scikit-learn.org (the pickle security warning; skops and ONNX alternatives).
20. DVC — Data Version Control. [Documentation](#).
21. ONNX — Open Neural Network Exchange. onnx.ai. Hugging Face, safetensors. [Documentation](#).
22. Gama, Zliobaite, Bifet, Pechenizkiy & Bouchachia, “A Survey on Concept Drift Adaptation,” *ACM Computing Surveys* 46(4), Article 44, 2014. [DOI:10.1145/2523813](https://doi.org/10.1145/2523813) (Figure 2 — patterns of change over time).
23. Quinonero-Candela, Sugiyama, Schwaighofer & Lawrence (eds.), *Dataset Shift in Machine Learning*, MIT Press, 2009. [Book](#).
24. Ribeiro, Wu, Guestrin & Singh, “Beyond Accuracy: Behavioral Testing of NLP Models with CheckList,” ACL 2020. [arXiv:2005.04118v1](https://arxiv.org/abs/2005.04118v1) (minimum functionality, invariance and directional tests).
25. Mitchell, Wu, Zaldivar, Barnes, Vasserman, Hutchinson, Spitzer, Raji & Gebru, “Model Cards for Model Reporting,” FAT* 2019. [arXiv:1810.03993v2](https://arxiv.org/abs/1810.03993v2).
26. Gebru, Morgenstern, Vecchione, Vaughan, Wallach, Daumé III & Crawford, “Datasheets for Datasets,” *Communications of the ACM* 64(12), 2021, pp. 86–92. [arXiv:1803.09010v8](https://arxiv.org/abs/1803.09010v8).
27. C. Huyen, *Designing Machine Learning Systems*, O’Reilly, 2022 — the recommended book-length treatment of everything in this deck.

28. A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed., O'Reilly, 2022, Chapter 2 and Chapter 19 (end-to-end project; training and deploying at scale).
29. Han, Pool, Tran & Dally, "Learning both Weights and Connections for Efficient Neural Networks," NeurIPS 2015. [arXiv:1506.02626](#) (magnitude pruning).
30. Han, Mao & Dally, "Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding," ICLR 2016. [arXiv:1510.00149](#).
31. Jacob et al., "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," CVPR 2018. [arXiv:1712.05877](#).
32. Wang & Spindler, "Model Quantization: Concepts, Methods, and Why It Matters," NVIDIA Technical Blog, 2025 (the affine and symmetric quantisation figure).
33. Hinton, Vinyals & Dean, "Distilling the Knowledge in a Neural Network," NIPS 2014 Deep Learning Workshop. [arXiv:1503.02531](#).
34. Sanh, Debut, Chaumond & Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," 2019. [arXiv:1910.01108](#).